HND in Computing Software Engineering Curriculum Overview

Program Overview:

The HND in Computing Software Engineering program at Computer College Minuwangoda is designed to provide students with a strong foundation in software development principles and practices. Through a combination of theoretical learning and practical hands-on experience, students will develop expertise in software design, development, testing, and maintenance.

Program Duration: 2 Years

Learning Outcomes:

Upon completion of the program, students will be able to:

- Apply software engineering principles and methodologies to develop robust and scalable software solutions.
- Design and implement software systems using various programming languages and development tools.
- Collaborate effectively in a team environment to deliver software projects on time and within budget.
- Test and debug software applications to ensure quality and reliability.
- Adapt to evolving technologies and industry trends in software engineering.
- Communicate technical concepts clearly and professionally to stakeholders.

Admission Requirements:

- High school diploma or equivalent qualification.
- Basic understanding of mathematics and computer science concepts.
- Proficiency in English language (speaking, reading, and writing).

Teaching Methodology:

The program employs a variety of teaching and learning methods, including:

- Lectures delivered by experienced faculty members.
- Practical coding sessions to reinforce programming concepts.
- Group projects and assignments to simulate real-world software development scenarios.
- Industry guest lectures and workshops on emerging technologies and best practices.
- Software development labs equipped with industry-standard tools and technologies.

Assessment Methods:

Students will be assessed through a combination of:

- Coding assignments and projects evaluating programming skills and problem-solving abilities.
- Presentations and demonstrations of software solutions.
- Software testing and debugging exercises.
- Peer and self-assessments of team-based projects.

Career Opportunities:

Graduates of the HND in Computing Software Engineering program can pursue various career paths, including:

- Software Developer
- Software Engineer
- Web Developer
- Mobile Application Developer
- Quality Assurance Engineer
- Technical Support Engineer

Year 1:

Introduction to Computing and Software Engineering

Overview of computing principles and software engineering concepts
Introduction to software development life cycle (SDLC)
Basics of programming paradigms and methodologies

Programming Fundamentals

Introduction to programming languages (e.g., Python, Java, C++)
Basic syntax, data types, and control structures
Problem-solving techniques and algorithm design

Mathematics for Computing

Mathematical foundations for software engineering
Discrete mathematics and logic
Algebra and calculus relevant to computing

Database Design and Development

Relational database concepts and design principles
SQL fundamentals for database manipulation
Database normalization and query optimization

Web Development

Introduction to web technologies (HTML, CSS, JavaScript)
Basics of client-side and server-side scripting
Web development frameworks and libraries

Software Development Tools and Environments

Integrated Development Environments (IDEs) and code editors
Version control systems (e.g., Git)
Software testing and debugging techniques

Year 2:

Object-Oriented Programming

Principles of object-oriented programming (OOP)

Advanced topics in inheritance, polymorphism, and encapsulation

Design patterns and best practices in OOP

Software Engineering Principles

Software requirements engineering and elicitation techniques

Software design methodologies (e.g., UML)

Software architecture and design patterns

Software Development Lifecycle

Agile development methodologies (e.g., Scrum, Kanban)

Iterative and incremental development practices

Continuous integration and deployment (CI/CD) pipelines

Advanced Programming Concepts

Data structures and algorithms

Advanced topics in programming languages (e.g., concurrency, parallel programming)

Performance optimization and profiling techniques

Software Quality Assurance

Testing methodologies (unit testing, integration testing, etc.)

Test-driven development (TDD) and behavior-driven development (BDD)

Software quality metrics and measurement techniques

Capstone Project

Real-world software engineering project

Requirements analysis, design, implementation, and testing phases

Documentation and presentation of project outcomes